Rešitve in dopolnitve predavanj

Rešitve in dopolnitve praznih mest iz predavanj

Opomba: originalnih PDF-jev nisem spreminjal. Ker direktno urejanje PDF-jev brez zanesljivega renderiranja ni varno, so spodaj rešitve organizirane po datoteki, temi in strani/prosojnici. To uporabi kot spremljevalni dokument ob prosojnicah.

`eaa263f7-76f0-4079-ad23-58eeea71b9a7.pdf` - uvod, zahtevnost

Stran 3 - zakaj merimo čas teoretično

- Čas izvajanja nas zanima, ker želimo primerjati algoritme neodvisno od konkretnega računalnika.
- Razlika med 1 ms in 2 ms postane pomembna pri velikem številu ponovitev ali velikih vhodih.
- V teoriji časa ne merimo v sekundah, ampak v številu osnovnih korakov kot funkcijo velikosti vhoda `n`.
- Če je `T(n)=C*f(n)`, pri asimptotiki konstante in nižje člene ignoriramo.

Strani 6-8 - kako veliko je veliko

Za grobo oceno velja:

- `O(log n)` in `O(n)` sta običajno zelo dobra.
- `O(n log n)` je praktičen za velika urejanja.
- `O(n^2)` hitro postane problem pri milijonih elementov.
- `O(2^n)` in `O(n!)` sta uporabna samo za zelo majhne `n`.

Mini trik: če v nalogi piše 1 GHz, računaj približno `10^9` osnovnih operacij/s.

`c9a3298c-761d-49b8-8af4-7cbe31c6282c.pdf` - ADT in osnovne strukture

Stran 5 - dinamična tabela

Smiselno povečevanje tabele: ob zapolnitvi povečamo kapaciteto za faktor, najpogosteje `2x`.

Zakaj ne povečujemo za `+1`?

- Pri `+1` bi ob skoraj vsakem vstavljanju kopirali celo tabelo: skupno `1+2+...+n = Θ(n^2)`.
- Pri podvajanju je skupno število kopiranj do `n` vstavljanj `O(n)`, zato je amortizirana cena `insert` enaka `O(1)`.

Stran 8 - tabela tabel

Za `n` elementov potrebujemo tabele velikosti `1, 2, 4, 8, ...`, podobno kot binarni zapis števila `n`.

- Zasedena je tabela `A_i`, če je bit `i` v binarnem zapisu `n` enak `1`.
- Število zasedenih tabel je `popcount(n)`.
- Največji indeks je ⌊`log2 n`⌋, zato je vseh možnih nivojev do največjega `O(log n)`.

- Iskanje: v vsaki zasedeni urejeni tabeli uporabimo binarno iskanje.
- Čas iskanja: `O(log^2 n)`, ker je tabel `O(log n)`, vsaka pa se išče z bisekcijo.
- Vstavljanje je kot pri binarnem števcu: če je `A_i` polna, jo zlijemo s prenosom v `A_{i+1}`. Amortizirano `O(log n)` ali bolje, odvisno od modela štetja kopiranj.

Stran 13 - slovar

- Če dvakrat vstavimo isti ključ, slovar tipično posodobi vrednost, ne shrani dveh kopij istega ključa.
- Slovar ima lahko samo ključe; takrat je praktično množica.

Strani 21-31 - preskočni seznam

Preskočni seznam je urejen seznam z dodatnimi kazalci za preskoke.

Postopek iskanja:

1. Začni na najvišjem nivoju v glavi seznama.
2. Dokler naslednji element ni prevelik, se pomikaj desno.
3. Ko ne moreš več desno, pojdi en nivo dol.
4. Končaš na nivoju `0`.

Naključna višina elementa:

- Element je vedno na nivoju `0`.
- Mečemo kovanec; dokler pada izbrani uspeh, element promoviramo za en nivo.
- Pri verjetnosti `1/2` je pričakovana višina `O(log n)`.
- Pričakovano iskanje in vstavljanje: `O(log n)`.

`b8a0228c-5230-4ccc-be50-eee727f2bc1c.pdf` - amortizirana analiza

Metoda vsote

Za zaporedje `n` operacij izračunamo skupno najslabšo ceno `T(n)`. Amortizirana cena ene operacije je `T(n)/n`.

Primer razširjen sklad:

- Vsak element je lahko enkrat dodan in največ enkrat odstranjen.
- Če naredimo `n` operacij, je skupno število realnih premikov elementov `O(n)`.
- Amortizirana cena `push`, `pop`, `multipop`: `O(1)`.

Primer dvojiški števec:

- Bit `A[0]` se spremeni pri vsaki operaciji.
- Bit `A[1]` pri vsaki drugi.
- Bit `A[i]` pri vsaki `2^i`-ti.
- Skupno število sprememb bitov je `n + n/2 + n/4 + ... < 2n`.
- Amortizirana cena povečanja: `O(1)`.

Metoda kopičenja

Ideja: poceni operacije plačajo malo več in kredit shranimo za drage operacije.

- Sklad: `push` plača 2 enoti; 1 za vstavljanje, 1 kredit za kasnejši izbris.
- Dvojiški števec: ob nastavitvi bita na 1 shranimo kredit za kasnejši preklop nazaj na 0.

Metoda potenciala

Amortizirana cena:

```
amortizirana = realna cena + Φ(D_i) - Φ(D_{i-1})
```

Pazi: funkcija potenciala mora biti izbrana tako, da ne ustvarjamo negativnega dolga. Tipično zahtevamo $\Phi(D_i) \geq \Phi(D_0)$.

`9789321c-64c3-45f0-a279-56d205df82ce.pdf` - drevesa

BST

- Iskanje/vstavljanje/brisanje: $O(h)$, kjer je h višina drevesa.
- Če je drevo uravnoteženo: $h=O(\log n)$.
- Če je degenerirano v seznam: $h=O(n)$.
- Minimum v BST je najbolj levo vozlišče, maksimum najbolj desno.

AVL

AVL pogoj: za vsako vozlišče je razlika višin levega in desnega poddrevesa največ 1.

Rotacije:

- LL: desna rotacija.
- RR: leva rotacija.
- LR: leva rotacija nad levim sinom, nato desna rotacija.
- RL: desna rotacija nad desnim sinom, nato leva rotacija.

Čas: iskanje, vstavljanje, brisanje $O(\log n)$.

B/B+ drevesa

- Uporabna za diske in baze, ker zmanjšajo število dostopov do blokov.
- Več ključev v vozlišču pomeni manjšo višino.
- B+ drevo hrani vse podatke v listih, notranja vozlišča so indeks.

Rdeče-črno drevo

Lastnosti:

- Vsako vozlišče je rdeče ali črno.
- Koren je črn.

- Rdeče vozlišče nima rdečega otroka.
- Vsaka pot od vozlišča do praznega lista ima enako število črnih vozlišč.

Čas: $O(\log n)$, ker je višina omejena z $O(\log n)$.

AVL je strožje uravnoteženo; RČ ima pogosto manj rotacij pri vstavljanju/brisanjih.

`41e289b2-60c5-48f4-b5f7-1599164cb827.pdf` - prioritetna vrsta, zgoščevanje, Bloom, DSU

Vrsta s prednostjo / kopica

- `min`: $O(1)$ pri min-kopici.
- `insert`: $O(\log n)$ zaradi popravljanja navzgor.
- `deleteMin`: $O(\log n)$ zaradi popravljanja navzdol.
- `heapify` na celotni tabeli: $O(n)$, ne $O(n \log n)$.

Zgoščena tabela

Veriženje:

- Indeks: $h(k)$.
- V košu hranimo seznam/BST elementov z istim indeksom.
- Pri dobri zgoščevalni funkciji in faktorju zasedenosti $\alpha=n/m$ je pričakovano iskanje $O(1+\alpha)$.

Odprto naslavljanje:

- Linearno: $h(k,i)=(h(k)+i) \bmod m$.
- Kvadratno ali dvojno zgoščevanje zmanjšata grupiranje.
- Brisanje potrebuje poseben označevalec `DELETED`, drugače prekinemo iskalne verige.

Bloomov filter

Odgovora sta:

- "zagotovo NE"
- "verjetno DA"

Lažno pozitiven odgovor je možen, lažno negativen pri navadnem Bloomovem filtru ni možen, če ne brišemo.

Brisanje:

- Navaden Bloomov filter ne zna varno brisati.
- Rešitev: counting Bloom filter s števci.

Disjunktne množice

Operacije:

- `makeSet(x)`

- `find(x)`
- `union(x,y)`

Učinkovita izvedba:

- gozd dreves,
- union by rank/size,
- path compression.

Amortizirano skoraj konstantno: $O(\alpha(n))$, kjer je α inverzna Ackermannova funkcija.

`4d7fa5ce-45de-4397-833e-bfead45e6b22.pdf` - dinamično programiranje

Fibonacci

Rekurzija brez shranjevanja:

- $T(n)=T(n-1)+T(n-2)+O(1)$
- eksponentno, približno $O(\phi^n)$.

Memoizacija ali tabulacija:

- vsako stanje izračunamo enkrat,
- čas $\Theta(n)$,
- prostor $\Theta(n)$ oziroma $\Theta(1)$, če hranimo samo zadnji dve vrednosti.

Verižno množenje matrik

Če ima M_i dimenzijo $d_{i-1} \times d_i$, je cena delitve pri k:

$$c(i,k) + c(k+1,j) + d_{i-1} * d_k * d_j$$

Rekurenca:

$$c(i,j)=\min_{i<=k<j}(c(i,k)+c(k+1,j)+d_{i-1}d_kd_j)$$

Osnova: $c(i,i)=0$.

Čas: $O(n^3)$, prostor: $O(n^2)$.

0/1 nahrbtnik

$dp[i][w]$ = najboljša vrednost s prvimi i predmeti in kapaciteto w.

Če predmeta i ne vzamem:

$$dp[i][w]=dp[i-1][w]$$

Če ga vzamem:

$$dp[i][w]=dp[i-1][w-w_i]+v_i$$

Vzamem maksimum. Čas `O(nV)` oziroma `O(nW)`, kjer je `W` kapaciteta.

`f9097b46-0b19-48a1-a360-83f8ae291bd8.pdf` - razredi kompleksnosti

- P: odločitveni problemi, rešljivi v polinomskem času.
- NP: rešitve znamo preveriti v polinomskem času.
- NP-poln: v NP in vsak problem iz NP se nanj polinomsko reducira.
- NP-težak: vsaj tako težak kot NP-polni, ni nujno v NP.

Pazi:

- Zahtevnost algoritma ni isto kot zahtevnost problema.
- Če poznamo eksponenten algoritem, to še ne pomeni, da problem ni v P.
- Če bi našli polinomski algoritem za en NP-poln problem, bi veljalo `P=NP`.

`2a7980df-6c3e-4fe2-aad2-fe12d98d310f.pdf` - grafi

Obdelane samo zahtevane teme: Prim, Kruskal, topološko urejanje, Bellman-Ford, Dijkstra, Welsh-Powell. Največji pretok je po navodilu izpuščen.

Prim

Uporabiš za MST v povezanem neusmerjenem uteženem grafu.

Postopek:

1. Začni v poljubnem vozlišču.
2. Vzdržuj množico že vključenih vozlišč.
3. Vedno dodaj najcenejšo povezavo iz drevesa do vozlišča zunaj drevesa.
4. Končaj po `n-1` povezavah.

Zahtevnost:

- matrika: `O(n^2)`,
- seznam sosedov + prioritetna vrsta: `O((n+m) log n)`.

Kruskal

Postopek:

1. Uredi povezave po teži.
2. Začni s praznim gozdom.
3. Povezavo dodaj, če ne ustvari cikla.
4. Za preverjanje komponent uporabi disjunktne množice.

Zahtevnost: `O(m log m)` zaradi urejanja povezav.

Topološko urejanje

Uporabiš samo za usmerjene aciklične grafe (DAG).

Kahnov algoritem:

1. Izračunaj vhodne stopnje.
2. V vrsto daj vozlišča z vhodno stopnjo 0.
3. Odstranjuj vozlišča in zmanjšuj stopnje naslednikov.
4. Če ne odstraniš vseh vozlišč, graf ima cikel.

Najkrajše poti v DAG:

1. Topološko uredi.
2. `d[start]=0`, ostalo ∞.
3. Sproščaj povezave v topološkem vrstnem redu.

Čas `O(n+m)`.

Dijkstra

Uporabiš, ko so vse uteži nenegativne.

Postopek:

1. `d[s]=0`, ostalo ∞.
2. Vedno izberi neobiskano vozlišče z najmanjšo trenutno razdaljo.
3. Sprosti njegove izhodne povezave.
4. Ko je vozlišče dokončno izbrano, se njegova razdalja ne bo več izboljšala.

Pazi: če vidiš negativno utež, Dijkstra odpade.

Bellman-Ford

Uporabiš pri negativnih utežeh ali ko želiš zaznati negativen cikel.

Postopek:

1. `d[s]=0`, ostalo ∞.
2. `n-1` krat sprosti vse povezave.
3. Še enkrat preveri vse povezave: če se kaj še izboljša, obstaja dosegljiv negativen cikel.

Čas `O(nm)`.

Welsh-Powell

Hevristika za barvanje grafa.

Postopek:

1. Vozlišča uredi padajoče po stopnji.
2. Vzemi prvo nepobarvano vozlišče in mu dodeli novo barvo.
3. Z isto barvo pobarvaj čim več naslednjih vozlišč, ki niso sosednja že pobarvanim s to barvo.

4. Ponavljaj, dokler niso pobarvana vsa vozlišča.

Rezultat ni nujno optimalen, je pa hiter in pogosto dober.